

Guia de Referência: 25 Funções Essenciais em Python

Dicionário de Funções Embutidas (Built-in)

28 de maio de 2026

Resumo

Para programar em Python com fluidez, desde a criação de scripts simples até o desenvolvimento de redes neurais, conhecer as funções embutidas (*built-in functions*) é essencial. Elas estão sempre disponíveis no escopo global, eliminando a necessidade de importar módulos externos. Este guia prático descreve 25 das funções mais importantes e amplamente utilizadas, organizadas por categorias lógicas de utilidade.

1 Entrada, Saída e Investigação

1. **print()**: Exibe informações na tela (no console). É a sua principal ferramenta para visualizar saídas e depurar o comportamento do código.
2. **input()**: Pausa a execução do programa e aguarda que o utilizador digite um dado no teclado, retornando esse valor como uma string.
3. **help()**: Invoca o sistema de ajuda integrado do Python. Ao passar um objeto, módulo ou função como argumento, ela exibe a sua documentação oficial.
4. **dir()**: Retorna uma lista com todos os atributos e métodos válidos associados a um determinado objeto, permitindo inspecionar o que ele “tem” e o que “sabe fazer”.

2 Tipos de Dados e Conversões

5. **type()**: Revela o tipo de dado de um objeto (se é uma string, um número inteiro, uma lista, etc.). Útil para validações e depuração de erros.
6. **isinstance()**: Verifica se um objeto pertence a um tipo específico ou a uma subclasse dele (ex: verifica se uma variável é uma lista), retornando **True** ou **False**.
7. **int()**: Converte ou trunca um valor (como um texto "10" ou um número decimal 3.14) para um número inteiro (10 e 3, respectivamente).
8. **float()**: Converte um número ou uma string de formato numérico compatível em um número decimal (ponto flutuante).
9. **str()**: Converte qualquer objeto ou valor na sua representação textual (string).
10. **list()**: Cria uma nova lista vazia ou converte uma coleção iterável (como uma tupla ou um texto) numa lista mutável.
11. **dict()**: Cria um dicionário, que é uma estrutura de dados flexível que armazena informações no formato de chave-valor.
12. **set()**: Cria um conjunto (set). Uma das suas propriedades mais importantes é remover automaticamente qualquer item duplicado da coleção recebida.

13. **tuple()**: Cria uma tupla, que se comporta como uma lista, porém de forma imutável (os seus valores não podem ser alterados após a criação).

3 Matemática Básica e Medidas

14. **len()**: Abreviação de *length*. Retorna a quantidade de itens presentes numa coleção (como o tamanho de uma lista ou o número de caracteres de um texto).
15. **sum()**: Realiza a soma aritmética de todos os elementos numéricos contidos dentro de um iterável (como uma lista ou tupla).
16. **min()**: Analisa uma coleção ou conjunto de argumentos e retorna o menor valor encontrado.
17. **max()**: Analisa uma coleção ou conjunto de argumentos e retorna o maior valor encontrado.
18. **abs()**: Retorna o valor absoluto (módulo) de um número, desconsiderando o sinal negativo (ex: `abs(-5)` resulta em 5).
19. **round()**: Arredonda um número decimal para o número inteiro mais próximo ou para uma quantidade específica de casas decimais definida pelo utilizador.

4 Manipulação e Organização de Coleções

20. **range()**: Gera uma sequência aritmética de números em formato imutável (ex: de 0 a 9). É massivamente utilizada para controlar o número de repetições em laços **for**.
21. **sorted()**: Recebe um iterável desorganizado e retorna uma *nova* lista contendo os mesmos elementos ordenados de forma crescente ou alfabética.
22. **enumerate()**: Recebe uma coleção e retorna um iterador que fornece pares contendo o índice posicional e o respetivo item (ex: 0: "Maçã", 1: "Banana").
23. **zip()**: Combina os elementos de duas ou mais coleções lado a lado, aglutinando-os em pares como um fecho de correr (junta o primeiro elemento de cada lista, depois o segundo, e assim por diante).

5 Programação Funcional (Transformações)

24. **map()**: Aplica uma função específica a cada um dos itens de uma coleção, gerando um novo iterador contendo os resultados dessas transformações.
25. **filter()**: Filtra os elementos de uma coleção com base numa função de teste. Retorna apenas os itens que atendem ao critério (ou seja, onde o teste retorna **True**).